

Day 9 - OOP Advanced: Inheritance, Dunder Methods, Composition

Day 9 — OOP Advanced: Inheritance, Dunder Methods, Composition

Objectives

- Understand inheritance: how one class can share and extend another's behavior
- Understand `super()` and why it exists
- Understand polymorphism — the same line of code behaving differently depending on what kind of object it's dealing with
- Meet the most important "dunder" (double-underscore) methods and what each hooks into
- Understand composition as an alternative to inheritance, and how to choose between them

Inheritance — one class building on another

Inheritance lets you define a new class that automatically gets all the attributes and methods of an existing class, and then adds or changes only what's actually different, instead of rewriting everything from scratch.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("subclasses must implement speak()")

    def introduce(self):
        return f"I am {self.name} and I say {self.speak()}"

class Dog(Animal):          # Dog INHERITS from Animal -- the parentheses show the parent class
    def speak(self):
        return "Woof"

class Cat(Animal):
    def speak(self):
        return "Meow"

rex = Dog("Rex")
print(rex.introduce())    # I am Rex and I say Woof
```

`Animal` here is called the **parent class** (or "base class," or "superclass"); `Dog` and `Cat` are **child classes** (or "subclasses"). Writing `class Dog(Animal):` means "`Dog` is a `Dog`, but it's *also* an `Animal`, and starts out with everything `Animal` already has." Notice `Dog` never wrote its own `__init__` or `introduce` method at all — it inherited both directly from `Animal`, completely free, and only needed to define what's genuinely different

about a `Dog`: its own version of `speak`.

`Animal.speak` deliberately raises `NotImplementedError` — this is a common, deliberate pattern meaning "every subclass of `Animal` MUST provide its own `speak` method; the base `Animal` class refuses to guess a default for you, since a generic `Animal` doesn't have a single specific sound." If you forgot to define `speak` in some new subclass, you'd find out immediately and clearly (a crash with a clear message), rather than silently getting some wrong, made-up default sound.

Polymorphism — the same call, different behavior, depending on the actual object

Notice something important in `introduce`: it calls `self.speak()` without knowing or caring in advance whether `self` is a `Dog`, a `Cat`, or some other `Animal` subclass someone writes in the future. Each type of object supplies its own correct answer when asked. **This is called polymorphism** ("many forms") — one piece of code (`self.speak()`, called from inside `introduce`) automatically does the *correct, different* thing depending on what kind of object it's actually attached to at the time, decided while the program is running, not decided in advance by the code itself.

This is genuinely powerful: you could write a hundred more `Animal` subclasses (`Bird`, `Cow`, `Snake`, anything), and `introduce` would keep working correctly for every single one, without you ever needing to go back and modify `introduce` itself — as long as each new subclass properly supplies its own `speak` method.

`super()` — reusing the parent's setup instead of duplicating it

If a subclass needs its *own* `__init__` (to accept and store some extra piece of data the parent doesn't have), but still wants the parent's original setup logic to run too, you shouldn't copy-paste that logic — you should call it directly via `super()`:

```
class Employee(Animal):    # a deliberately silly example, purely to show the mechanism clearly
    def __init__(self, name, salary):
        super().__init__(name)    # this literally runs Animal.__init__(self, name) for you
        self.salary = salary

e = Employee("Ana", 50000)
print(e.name, e.salary)    # Ana 50000
```

Without that `super().__init__(name)` line, `self.name` would never actually get set at all inside `Employee` — you'd have to manually retype `self.name = name` yourself, duplicating logic that already exists in `Animal`. And if `Animal.__init__` ever changed later (say, it started validating that `name` isn't empty), any subclass that had copy-pasted the old logic instead of calling `super()` would silently miss that update. Calling `super()` keeps the parent's setup as the single, authoritative source of truth, referenced rather than duplicated.

`isinstance()` versus `type()` — and why `isinstance` is almost always the right choice

```
isinstance(rex, Dog)    # True -- rex genuinely is a Dog
isinstance(rex, Animal) # True -- rex is ALSO an Animal, because Dog inherits from Animal
```

```
type(rex) == Animal          # False -- rex's EXACT type is Dog, not Animal, even though it IS an Animal
```

`isinstance` asks "is this object a Dog, OR anything that inherits from Dog?" — a broader, inheritance-aware question. `type(x) == SomeClass` asks a much narrower, stricter question: "is this object's exact type precisely `SomeClass`, no more, no less?" Prefer `isinstance` in almost every situation, because it keeps working correctly even as your codebase grows — code that checks `isinstance(x, Animal)` will happily keep working if you add a brand-new `Bird(Animal)` subclass next month; code that checks `type(x) == Dog` specifically will silently, incorrectly exclude that new `Bird`, even though a `Bird` genuinely is an `Animal` too.

Dunder methods — how your own classes hook into Python's built-in syntax

You've already used one dunder method: `__init__`. "Dunder" methods (from "double underscore," since they're written as `__name__`) are Python's mechanism for letting your own custom classes work naturally with built-in operators and syntax (`+`, `==`, `len(...)`, `print(...)`, `for...in`, and more), instead of forcing you to invent a differently-named method for every single class you write.

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"    # unambiguous text representation, mainly for developers

    def __str__(self):
        return f"({self.x}, {self.y})"         # friendlier text representation, for regular users

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)          # (4, 6) -- Python automatically calls v1.__add__(v2) when it sees the +
print(v1 == Vector(1, 2)) # True -- Python calls v1.__eq__(...) when it sees ==
```

`__repr__` versus `__str__`: if a class defines only one of them, that one is used everywhere. If a class defines both, `print(instance)` and `str(instance)` use `__str__` (meant to be nicely readable for an end user), while typing the instance's name directly into the interactive REPL, or calling `repr(instance)` explicitly, uses `__repr__` (meant to be precise and useful for a *developer* debugging the program — ideally something close to what you'd type back into Python to recreate an identical object). As a habit worth starting now: **always write a `__repr__` for classes you'll be debugging**, since without one, printing an instance just shows an unhelpful default like `<__main__.Vector object at 0x000001A2B3C4D5E6>`, which tells you almost nothing useful.

A few other dunder methods worth being able to recognize when you see them in other people's code, even before you've used them yourself: `__len__` (lets `len(your_object)` work), `__getitem__` (lets `your_object[index]` work), `__iter__` (lets `for item in your_object:` work), `__contains__` (lets `item in your_object` work), and `__enter__`/`__exit__` (the exact mechanism behind `with`, from Day 5 — now

you know what makes that work under the hood).

Composition — an alternative to inheritance for "has-a" relationships

Inheritance models an "is-a" relationship: a **Dog** is an **Animal**. **Composition** instead models a "has-a" relationship: one class simply holds an *instance* of another class as one of its own attributes, rather than inheriting from it at all:

```
class Engine:
    def start(self):
        return "Vroom"

class Car:
    def __init__(self):
        self.engine = Engine()      # Car HAS-A Engine -- it holds one as an attribute, it doesn't inherit

    def start(self):
        return self.engine.start()  # Car "delegates" the actual work to its Engine

my_car = Car()
print(my_car.start())  # Vroom
```

Why would you choose composition over inheritance, when either would technically work? Inheritance creates a fairly rigid, tightly-coupled relationship — a subclass depends on its entire parent's internal design, and if you ever needed a **Car** that used a totally different kind of engine (an **ElectricEngine**, say), inheritance would force you into an awkward class hierarchy to express that. Composition is far more flexible: swapping **Engine()** for **ElectricEngine()** inside **Car.__init__** requires no change at all to the **Car** class's overall shape, and doesn't risk breaking anything else that happens to inherit from **Car**.

A widely-repeated piece of professional advice, worth remembering as you start designing your own classes: **"favor composition over inheritance."** A reasonable starting rule of thumb: reach for inheritance when there's a genuine "is-a" relationship *and* you specifically want polymorphism (treating several different subclasses interchangeably through one shared interface, exactly like **Animal/Dog/Cat** above). Reach for composition when you're simply reusing a separate piece of functionality without needing that interchangeability.

Exercises

Open **exercises.py**, fill in each **# TODO**, and run **python exercises.py**.